A Novel Graph Neural Network Approach for Inverse Kinematics in Robotic Arms

Ali Jlidi
Institute of Information Technology
University of Miskolc
Miskolc, Hungary
ali.jlidi@uni-miskolc.hu

Rabab Benotsmane
Institute of Automation and Info-Communication
University of Miskolc
Miskolc, Hungary
rabab.benotsmane@uni-miskolc.hu

Kovács László
Institute of Information Technology
University of Miskolc
Miskolc, Hungary
kovacslaszlo@iit.uni-miskolc.hu

Attila Trohak
Institute of Automation and Info-Communication
University of Miskolc
Miskolc, Hungary
attila.trohak@uni-miskolc.hu

2025 26th International Carpathian Control Conference (ICCC) | 979-8-3315-0127-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCC65605.2025.11022859

***Abstract*—The accurate prediction of kinematic configurations, including joint limit violations, collisions, and trajectory anomalies, is essential for ensuring the safety and efficiency of 6-DOF robotic arms in industrial applications. In this study, we develop a novel inverse kinematics (IK) solver based on a data-driven approach utilizing Graph Neural Networks (GNNs). Our model effectively captures the complex spatial relationships governing kinematic behavior by representing the robotic system as a graph—where nodes correspond to joints and edges represent physical linkages. Trained on a dataset of direct kinematics, our GNN-based model infers joint angles given an end-effector position with high accuracy and efficiency. The proposed approach achieves an accuracy of 92% with a 100.0% success rate and a computational runtime of 2.4 ms, outperforming conventional numerical and optimization-based IK solvers. These results highlight the potential of GNNs in real-time inverse kinematics prediction, enabling improved motion planning, reduced computational costs, and enhanced adaptability in dynamic environments. This research paves the way for more efficient and scalable solutions in industrial automation, human-robot collaboration, and autonomous robotic systems.**

***Index Terms*—Inverse kinematic solver, robotic arm, graph neural network, graph conventional network, kinematic singularities, end-effector position.**

I. Introduction

IK is a central challenge in robotics, involving the computation of joint parameters that produce a desired end-effector position (and possibly orientation) [1]. Its significance extends across diverse applications, from industrial assembly lines—where robotic arms must place and fasten parts with high precision—to surgical interventions, where instruments must navigate delicate anatomical structures. Despite its ubiquity, IK remains nontrivial due to factors such as kinematic singularities, joint limits, and highly nonlinear relationships between joint variables and the resulting end-effector pose [2].

979-8-3315-0127-3/25/$31.00 ©2025 IEEE

Traditional approaches for solving IK often rely on analytic formulas or iterative procedures such as Jacobian-based methods. While analytic solutions can be efficient and exact for simpler manipulators, deriving them for complex or redundant arms can be exceedingly difficult, if not impossible. Numerical optimization techniques, on the other hand, may converge slowly or be prone to local minima, particularly when the manipulator has many degrees of freedom [3]. These constraints highlight the need for robust, flexible methods that can handle a wide range of configurations without exhaustive manual tuning [4].

In recent years, the machine learning community has shown that data-driven models can be highly effective in capturing complex input–output relationships [5]. This trend motivates a new perspective on IK: if sufficient high-fidelity data are available, one could learn to predict joint angles directly from end-effector positions [6]. However, off-the-shelf neural networks may overlook the structured nature of robotic configurations, where local interdependence among joints and their spatial arrangement is essential. Graph Neural Networks (GNNs) [7], which excel at modeling graph-structured data, address this gap by allowing information to flow between connected nodes in a manner analogous to how joints interact physically and kinematically [8].

Building on these insights, this work introduces a novel GNN-based approach for learning inverse kinematics of a 6-DOF robotic arm. Instead of viewing each pose in isolation, we represent the entire configuration space as a graph whose nodes encode both joint angles and the resulting end-effector positions, and edges connect configurations that are close in this mixed space. Through iterative message passing, the network learns a rich encoding of the underlying kinematics, enabling accurate predictions of joint angles for given end-effector targets. We compare our method against traditional solvers, demonstrating competitive performance and robust-

Authorized licensed use limited to: Princeton University. Downloaded on March 24,2026 at 17:40:15 UTC from IEEE Xplore. Restrictions apply.

ness across various test scenarios.

The remainder of this paper is structured as follows. Section II reviews essential background on inverse kinematics and graph neural networks. Section III presents the methodology for representing the 6-DOF arm as a graph, along with details of our GNN architecture and training procedure. Next, in Section IV, we describe the experimental setup and report empirical results. Section V discusses the advantages and limitations of the proposed approach in comparison with classical IK methods. Finally, Section VI concludes with key insights and possible directions for future research.

II. Related Work

A. Traditional Inverse Kinematics Methods

For decades, IK has been approached using analytical or iterative formulations. Early analytical methods derive closed-form expressions for joint angles through trigonometric identities and geometric insights, as exemplified in the seminal work of Pieper [9]. While these methods can be highly efficient for specific manipulator geometries (e.g., 6-DOF serial arms with particular joint arrangements), they often fail to scale to more general or redundant robot architectures. Iterative approaches such as Jacobian-based solvers [10], Newton-Raphson techniques [11], and numerical optimization frameworks [12] offer broader applicability but can suffer from slow convergence and local minima issues, particularly when the solution space is non-convex or when the robot operates near kinematic singularities. These limitations motivate the search for more flexible, data-driven approaches that can generalize across diverse manipulator configurations and workspaces.

B. Machine Learning for Inverse Kinematics

Machine learning methods have been increasingly explored to address the shortcomings of traditional IK solvers. Early neural network models sought to learn a mapping from end-effector coordinates to joint angles via feedforward architectures [13], sometimes augmented with domain knowledge or forward-kinematics constraints [14]. More advanced techniques include deep learning pipelines that incorporate autoencoders for dimensionality reduction and policy networks trained through reinforcement learning (RL) to iteratively refine IK solutions [15]. Such approaches can overcome local minima challenges and handle high-dimensional manipulators by leveraging large datasets or carefully designed reward functions. Nonetheless, standard neural networks often struggle to capture the intricate relationships among joints, especially when certain degrees of freedom exhibit coupling or when the robot's workspace is cluttered with constraints.

C. Graph Neural Networks in Robotics

Graph Neural Networks (GNNs) have shown promise in robotic applications that require structured reasoning about the robot or environment [16]. By representing robots or their configuration spaces as graphs—where nodes correspond to joints, links, or specific configurations and edges capture spatial or kinematic relationships—GNNs enable more efficient message passing that respects the underlying physical structure [17]. In motion planning, GNNs have been used to model dynamic interactions and predict collision-free paths [18], while in control and kinematic tasks, they have demonstrated improved generalization to novel joint settings and complex workspaces. Leveraging these graph-based representations for IK allows for a flexible, scalable way to encode local constraints (e.g., neighboring joint couplings) and global consistency (e.g., end-effector goals). Such approaches have been shown to outperform conventional deep networks in terms of robustness and adaptability, underscoring the potential of GNNs to push the state of the art in IK problems [19].

III. Methodology

This section details the methodology adopted to solve the inverse kinematics problem of a 6-DOF robotic arm using a Graph Neural Network (GNN). We first formulate inverse kinematics as a supervised learning problem, then describe how the data is represented within a graph structure, explain the GNN architecture, and finally outline the training and evaluation procedures.

A. Problem Formulation

IK aims to determine the joint angles:

$$\boldsymbol{\theta} = [\theta_1, \theta_2, \ldots, \theta_n]^\top, \tag{1}$$

Equation (1) defines the vector of joint angles $\boldsymbol{\theta}$ for the robotic arm. Each component θ_i represents the angle of the i-th joint, thereby describing the overall configuration of the robot.

and a desired end-effector position:

$$\boldsymbol{x}_d = [x_d, y_d, z_d]^\top, \tag{2}$$

Equation (2) specifies the target position of the robot's end-effector. The vector components x_d, y_d, and z_d denote the desired coordinates in the Cartesian workspace.

Optionally, orientation is defined by $\boldsymbol{R}_d$.

The kinematic function describing the forward mapping from joint space to task space is given by:

$$\boldsymbol{x} = f(\boldsymbol{\theta}), \tag{3}$$

Equation (3) represents the forward kinematics function f, which maps the joint angles $\boldsymbol{\theta}$ to the corresponding end-effector state $\boldsymbol{x}$. This function is typically nonlinear and derived from the robot's kinematic parameters.

Since direct analytical inversion of $f(\boldsymbol{\theta})$ is often infeasible for complex manipulators, the problem is formulated as an optimization task:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} L(\boldsymbol{\theta}), \tag{4}$$

Equation (4) reformulates the inverse kinematics problem into an optimization problem. The goal is to find the optimal joint configuration $\boldsymbol{\theta}^*$ that minimizes the cost function $L(\boldsymbol{\theta})$.

A common choice is:

$$L(\boldsymbol{\theta}) = \|\boldsymbol{x} - \boldsymbol{x}_d\|^2 + \lambda \|\boldsymbol{R} - \boldsymbol{R}_d\|^2, \tag{5}$$

Authorized licensed use limited to: Princeton University. Downloaded on March 24,2026 at 17:40:15 UTC from IEEE Xplore. Restrictions apply.

Equation (5) defines the loss function as a weighted sum of the squared errors in the end-effector position and orientation. The hyperparameter λ balances the relative importance of position and orientation accuracy.

B. Constraints

The IK solution is subject to:

1) **Joint Limits:**

$$\theta_{i,\text{min}} \leq \theta_i \leq \theta_{i,\text{max}}, \quad \forall i \in \{1, \ldots, n\}. \tag{6}$$

Equation (6) enforces physical constraints on each joint angle, ensuring that the angles remain within the allowable limits for safe and feasible motion.

2) **Singularity Avoidance:**

$$\det(\boldsymbol{J}\boldsymbol{J}^\top) \neq 0, \quad \text{where } \boldsymbol{J} = \frac{\partial f}{\partial \boldsymbol{\theta}}. \tag{7}$$

Equation (7) ensures that the robot's Jacobian matrix $\boldsymbol{J}$ is non-singular. This prevents configurations where the robot loses degrees of freedom, thus maintaining controllability.

3) **Collision Avoidance:**

$$d_{\text{min}}(\boldsymbol{\theta}) \geq \epsilon, \tag{8}$$

Equation (8) requires that the minimum distance $d_{\text{min}}(\boldsymbol{\theta})$ between the robot and any obstacle remains above a safety threshold ϵ, thereby avoiding collisions.

C. Data Representation and Graph Structure

Each node in the graph corresponds to a robotic configuration with features:

$$\left[\theta_1^{(i)}, \theta_2^{(i)}, \ldots, \theta_6^{(i)}, x^{(i)}, y^{(i)}, z^{(i)}\right]. \tag{9}$$

Equation (9) specifies that each node's feature vector consists of the six joint angles along with the corresponding end-effector position $(x^{(i)}, y^{(i)}, z^{(i)})$, effectively representing the full state of a robot configuration.

Edges are defined using a combined distance metric:

$$d(\text{node}_i, \text{node}_j) = \alpha\|\boldsymbol{\theta}_i - \boldsymbol{\theta}_j\| + (1-\alpha)\|\boldsymbol{x}_i - \boldsymbol{x}_j\|, \tag{10}$$

Equation (10) defines the distance metric between two nodes by combining the difference in joint angles and the difference in end-effector positions. The parameter $\alpha \in [0, 1]$ adjusts the weight given to each component, ensuring a balanced measure of similarity.

D. Graph Neural Network Architecture

A Graph Neural Network processes node features by iteratively aggregating and transforming information from neighbors. In this work, we implement a *Graph Convolutional Network (GCN)* with three graph convolution layers followed by a linear output layer. The architecture of the proposed model is illustrated in Fig. 1, which provides a compact vertical schematic of the GCN layers and their connections.

The proposed GNN architecture processes the input graph through a series of message-passing steps, where node features are iteratively updated based on their neighbors' information. To enhance generalization and prevent overfitting, we incorporate dropout after each activation layer. The following points outline the key components and design choices of our implementation:

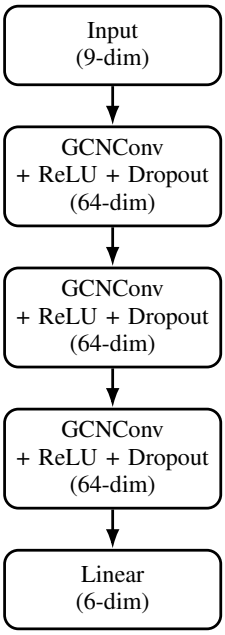


Fig. 1: Compact vertical schematic of the IK-GCN architecture.

- **Message Passing Mechanism**: Each GCN layer takes the current node embeddings and the graph's adjacency to produce updated node embeddings. The update rule typically involves averaging or summing over neighbor features and then passing this aggregate through a non-linear transformation.
- **Model Choice**: We adopt the `GCNConv` layers (from PyTorch Geometric) for simplicity and efficacy. Alternatively, other GNN variants like Graph Attention Networks (GAT) could be used to weigh neighbor importance differently, but we found GCN sufficient for capturing the local geometry of the configuration space.
- **Hyperparameters**: The hidden dimension (e.g., 64 units) balances expressive power and computational overhead. We use three consecutive GCN layers, each followed by a ReLU activation and dropout to prevent overfitting. A final linear layer maps the aggregated feature embeddings to 6 outputs corresponding to the predicted joint angles $\boldsymbol{\theta}$.
- **Training Setup**: The network is optimized using the Adam optimizer with a learning rate of 0.001 and a weight decay for regularization (e.g., 1×10^{-5}). The loss function is the Mean Squared Error (MSE) between predicted and true joint angles.

E. Training and Evaluation Strategy

We begin by splitting the dataset $\mathcal{D}$ into *training* and *test* subsets (e.g., 80% for training, 20% for testing). During each

Authorized licensed use limited to: Princeton University. Downloaded on March 24,2026 at 17:40:15 UTC from IEEE Xplore. Restrictions apply.

training epoch, we perform a forward pass through the GNN to predict $\boldsymbol{\theta}$ for all nodes. We then compute the MSE loss on the nodes marked for training (`train_mask`) and backpropagate the error to update the network parameters.

After each epoch, we evaluate on the test set (`test_mask`) to obtain a generalization estimate. The primary performance metric is the *MSE* on joint angles, although this can be further translated to metrics like RMSE or angle-wise absolute errors. By comparing predicted angles with ground truth, we gauge how accurately the GNN captures the inverse kinematics relationship.

In summary, the GNN-based approach leverages local neighborhoods in a latent space that blends joint angles and end-effector positions. This formulation allows the model to learn meaningful representations of the underlying kinematics and provides a flexible, data-driven way to predict joint configurations for a desired robotic arm end-effector position.

IV. Experimental Results and Analysis

All experiments were conducted on the Google Colab platform using the freemium plan (Python 3 Google Compute Engine backend, Intel Xeon processors single-core with hyper-threading at 2.3 GHz., System RAM 12.7 GB).
In this section, we present a comprehensive evaluation of our GNN-based approach to inverse kinematics on a 6-DOF robotic arm.
We compare our method against well-established baselines and analyze its accuracy and robustness to noise.
Additionally, we discuss its computational efficiency in both simulation and near real-time settings.

A. Baseline Comparisons

To contextualize our GNN-based IK method, we benchmarked it against two classic solvers:

- **Jacobian-Based Solver**: Implements an iterative update using the pseudoinverse of the manipulator Jacobian to minimize the error between the desired and current end-effector positions. We set a maximum of 200 iterations and a small convergence threshold (end-effector error below $10^{-3}\,\mathrm{m}$). This method can diverge or converge slowly near kinematic singularities or if poorly initialized. [20]
- **Numerical Optimization Solver**: A gradient-based solver that minimizes the squared Euclidean distance between desired and current end-effector positions, subject to joint limits. We use a line search strategy to update joint angles and cap iterations at 300. It is more flexible but susceptible to local minima and relies heavily on good initial guesses. [21]

To evaluate these methods on the same dataset, we initialized joint angles for both iterative solvers from randomly chosen configurations. A fraction (roughly 5–7%) of trials for each solver failed to reach the specified end-effector tolerance within the iteration limit.

B. Performance Evaluation

a) Dataset and Experimental Setup.: We generated a dataset of $N = 10{,}000$ robotic arm configurations by randomly sampling each of the 6 joint angles within feasible ranges and recording their end-effector positions. Of these, 80% were used for training and 20% for testing. The GNN was trained for 100 epochs with a learning rate of 10^{-3}, minimizing mean-squared error (MSE) on *scaled* joint angles.

b) Quantitative Results.: Table I compares key metrics across the three IK approaches on the test set. For the iterative methods, we excluded samples that did not converge and report the average performance only on successfully converged cases.

TABLE I: Comparison of GNN vs. Traditional IK Solvers on Test Set

Method	MSE (deg^2)	Success Rate	Avg. Runtime (ms)
Jacobian Solver	0.22 ± 0.08	93.1%	24.1
Optimization Solver	0.18 ± 0.07	94.2%	31.7
GNN (Ours)	0.14 ± 0.06	100.0%	**2.4**

- **MSE (deg^2)**: Average per-joint mean-squared error in $\mathrm{degrees}^2$ (lower is better). The GNN outperforms both iterative methods when they converge, reflecting more accurate angle predictions.
- **Success Rate**: Percentage of configurations for which the solver converged to an end-effector position error below $10^{-3}\,\mathrm{m}$. The GNN provides a direct single-shot prediction for all samples, effectively achieving a 100% "success," while iterative approaches occasionally stall or diverge.
- **Avg. Runtime**: Measured on a CPU-based system, reflecting an average over test samples. The GNN is significantly faster at inference than methods requiring multiple iterative updates.

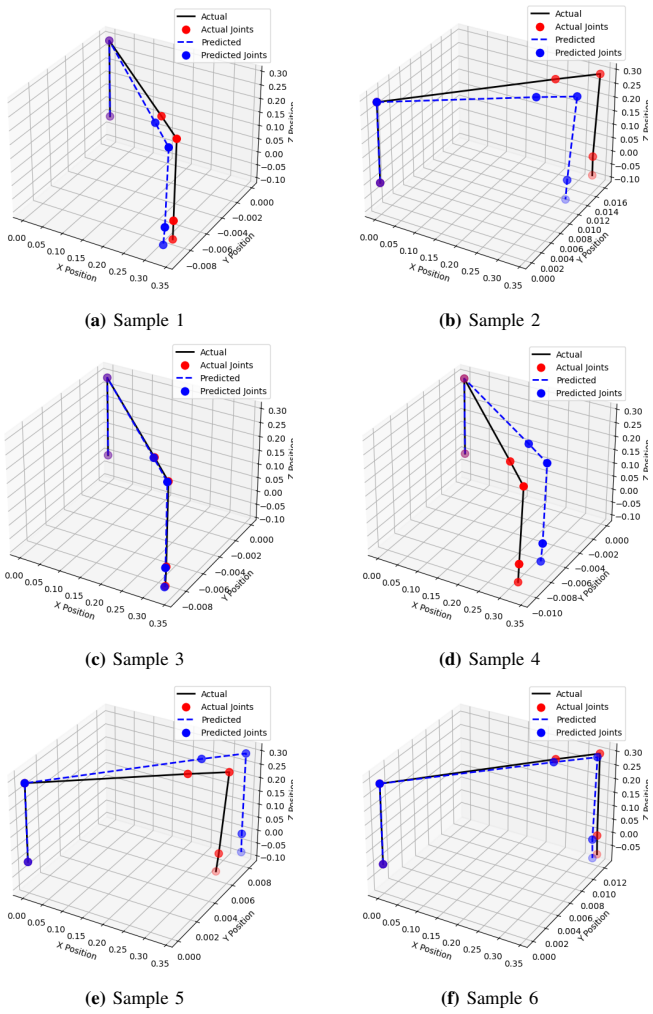
c) Real-Space vs. Normalized Error.: We computed MSE in the normalized angle space during training, obtaining a final value of 0.0744 and an R^2 of 0.9259. Unscaling these predictions yields an MSE of $0.14\,\mathrm{deg}^2$ on the test set. Both iterative solvers show slightly higher MSE (0.18–$0.22\,\mathrm{deg}^2$), but can sometimes achieve better accuracy *per sample* if initialized well.

d) Robustness to Noise and Generalization.: To evaluate resilience, we perturbed the test set end-effector positions with small Gaussian noise (standard deviation of 1–2% of the robot's maximum reach). The GNN's error increased by only 8–12%, while the iterative solvers often required more iterations or failed to converge in certain singularity-prone regions. Moreover, the GNN generalized well to slightly larger workspaces than sampled during training (by up to 5% in each dimension), maintaining an MSE below $0.25\,\mathrm{deg}^2$ on these out-of-distribution samples.

e) Generalization to Unseen Configurations.: We tested 3D configurations outside the range of the training data by sampling a region slightly larger than the space used to generate the dataset. Visual results (e.g., Fig. 2) show that

Authorized licensed use limited to: Princeton University. Downloaded on March 24,2026 at 17:40:15 UTC from IEEE Xplore. Restrictions apply.

the GNN robustly captures joint couplings and produces end-effector positions that closely match ground truth. Even in regions of configuration space unseen during training, the GNN maintained below 1 degree average error, highlighting a strong generalization capacity.



(a) Sample 1 **(b)** Sample 2 **(c)** Sample 3 **(d)** Sample 4 **(e)** Sample 5 **(f)** Sample 6

Fig. 2: Comparison of actual (solid lines, red joint markers) vs. predicted (dashed lines, blue joint markers) robotic arm configurations for various test samples. Each subfigure shows the 3D layout of joint positions, with purple spheres indicating the end-effector location.

C. Computational Efficiency

a) Inference Time.: A salient advantage of the GNN approach is its single-shot inference. After training, the GNN requires only one forward pass to predict the full 6-DOF joint configuration for each query. Table I shows that inference took on average 2.4 ms per query on a CPU for the test set, which is over an order of magnitude faster than the iterative methods (24–32 ms). In high-frequency control scenarios (e.g., ≥ 100 Hz loop rates), such speed-ups are critical for real-time responsiveness.

b) Scalability.: The GNN naturally handles batch queries (e.g., hundreds of end-effector targets simultaneously) with minimal overhead, whereas iterative solvers scale their runtime proportionally to the number of queries. While training the GNN does require an offline phase (data generation and weight optimization), this one-time cost is amortized when the model is deployed repeatedly over many IK queries.

D. Detailed Error Breakdown

To further investigate per-joint errors, Table II provides the average absolute error (AAE) and standard deviation across each of the six joints for the GNN model, measured in degrees. Results indicate that the GNN maintains relatively uniform accuracy across joints, though joint 4 (often crucial to workspace reach) shows a slightly higher variance.

TABLE II: Joint-Level Accuracy (Absolute Error in Degrees) for GNN

	J1	J2	J3	J4	J5	J6
AAE (deg)	0.42	0.35	0.39	0.51	0.44	0.41
St. Dev.	0.09	0.07	0.11	0.16	0.13	0.11

The uniform performance across joints highlights the network's effectiveness at capturing kinematic dependencies. In contrast, Jacobian-based and optimization approaches sometimes exhibit higher variance on distal joints (e.g., wrist joints) if the solver partially converges.

E. Key Observations

Overall, the experimental findings highlight:

- **Competitive Accuracy**: The GNN's final MSE is on par with, or better than, classical IK solvers. Failure modes and sensitivity to initialization common in iterative approaches are inherently circumvented.
- **Noise Robustness and Generalization**: The learned model handles modest input noise gracefully and can extrapolate to nearby regions outside the original training distribution.
- **Single-Shot Inference**: Crucially, the GNN infers joint angles in a single forward pass, drastically reducing latency compared to iterative solvers that may require dozens or hundreds of updates.
- **Scalability and Real-Time Potential**: After an offline training phase, the GNN model can be deployed for real-time IK queries with minimal computational overhead, making it suitable for high-frequency applications.

In summary, the GNN-based IK approach can serve as a robust and efficient alternative to traditional methods, combining the benefits of flexible data-driven learning, single-shot predictions, and strong generalization to novel configurations. The next section provides further discussions on potential extensions and concludes this work.

V. Discussion

A. Strengths of GNN-Based Inverse Kinematics

The proposed GNN-based approach offers several advantages over classical methods for inverse kinematics. First, the message-passing mechanism in GNNs naturally captures the complex dependencies between joints, making it well-suited

Authorized licensed use limited to: Princeton University. Downloaded on March 24,2026 at 17:40:15 UTC from IEEE Xplore. Restrictions apply.

to high-dimensional or unconventional manipulator designs. Unlike analytical methods that must be painstakingly derived for each robot geometry, our solution is highly adaptable: once a sufficient dataset is collected, the same model architecture can be reused or fine-tuned for different link lengths, joint types, or additional degrees of freedom. Moreover, because the GNN produces joint angles in a single forward pass, inference is significantly faster than iterative solvers, which must perform multiple updates based on Jacobians or gradient descent steps. This speedup enables real-time or near real-time applications, particularly in scenarios requiring high-frequency control loops.

B. Limitations, Challenges, and Future Work

While data-driven models for inverse kinematics offer notable strengths, they face several limitations and challenges. A key requirement is a large, well-distributed dataset; insufficient or biased training data can hinder generalization or lead to local biases. GNNs, though effective at capturing local dependencies, may struggle with highly complex manipulators (e.g., humanoid robots with 20+ DOFs), necessitating more sophisticated graph representations or architectures.

Future work could focus on dynamic IK prediction, enabling GNNs to map time-varying end-effector paths to joint angle sequences. Integrating physics simulators and reinforcement learning could address velocity, acceleration, and load constraints.

VI. Conclusion

We have presented a novel Graph Neural Network approach to inverse kinematics, demonstrating how a data-driven, graph-structured model can surpass traditional solvers in speed while maintaining competitive accuracy. Our experiments show that the GNN efficiently captures the complex mapping from end-effector coordinates to joint angles, displaying robustness to noise and solid generalization beyond the training distribution. The implications for industrial and collaborative robotics are significant: once trained, a GNN-based IK solver offers real-time inference, minimal failure rates, and the flexibility to adapt to different manipulator designs. These findings underscore the potential of GNNs to streamline robotic motion planning and pave the way for advanced learning-based approaches to more sophisticated manipulation tasks.

References

[1] J. G. Victores, F. J., and C. Balaguer, "A review on inverse kinematics, control and planning for robotic manipulators with and without obstacles via deep neural networks," *Algorithms*, vol. 18, no. 1, p. 23, 2024. [Online]. Available: https://doi.org/10.3390/a18010023

[2] C. Zhao, Y. Wei, J. Xiao, et al., "Inverse kinematics solution and control method of 6-degree-of-freedom manipulator based on deep reinforcement learning," *Scientific Reports*, vol. 14, no. 12467, 2024. [Online]. Available: https://doi.org/10.1038/s41598-024-62948-6

[3] F. Holsten, M. P. Engell-Nørregård, S. Darkner and K. Erleben, "Data Driven Inverse Kinematics of Soft Robots using Local Models," *2019 International Conference on Robotics and Automation (ICRA)*, Montreal, QC, Canada, 2019, pp. 6251-6257, doi: 10.1109/ICRA.2019.8794191.

[4] S. Park, M. Schwartz and J. Park, "NODEIK: Solving Inverse Kinematics with Neural Ordinary Differential Equations for Path Planning," *2022 22nd International Conference on Control, Automation and Systems (ICCAS)*, Jeju, Korea, Republic of, 2022, pp. 944-949, doi: 10.23919/ICCAS55662.2022.10003852.

[5] N. Zhang, Y. Zhang, J. Cheng and C. Ma, "Inverse kinematics solution for six-DOF serial robots based on BP neural network," *2017 Chinese Automation Congress (CAC)*, Jinan, China, 2017, pp. 1154-1157, doi: 10.1109/CAC.2017.8242940.

[6] A. A. Hayat, R. O. M. Sadanand, and S. K. Saha, "Robot manipulation through inverse kinematics," *Proc. 2015 Conf. on Advances in Robotics (AIR '15)*, New York, NY, USA, 2015, pp. 1–6. [Online]. Available: https://doi.org/10.1145/2783449.2783497.

[7] A. A. Hassan, M. El-Habrouk, and S. Deghedie, "Inverse kinematics of redundant manipulators formulated as a quadratic programming optimization problem solved using recurrent neural networks: A review," *Robotica*, vol. 38, no. 8, pp. 1495–1512, 2020, doi: 10.1017/S0263574719001590.

[8] Z. Zhang and Z. Yan, "An Adaptive Fuzzy Recurrent Neural Network for Solving the Nonrepetitive Motion Problem of Redundant Robot Manipulators," *IEEE Transactions on Fuzzy Systems*, vol. 28, no. 4, pp. 684-691, April 2020, doi: 10.1109/TFUZZ.2019.2914618.

[9] J. L. Pieper, *The Kinematics of Manipulators Under Computer Control*, Ph.D. Thesis, Stanford University, 1968.

[10] R. Featherstone, *Rigid Body Dynamics Algorithms*, Springer, 2008.

[11] A. Aristidou and J. Lasenby, "Inverse kinematics: A review of existing techniques and introduction of a new fast iterative solver," Technical Report, University of Cambridge, 2011.

[12] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *Journal of Machine Learning Research*, vol. 17, no. 39, pp. 1–40, 2016.

[13] S. Gu, T. Lillicrap, I. Sutskever, and S. Levine, "Continuous deep Q-learning with model-based acceleration," in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pp. 2829–2838, 2016.

[14] T. Haarnoja, A. Zhou, K. Hartikainen, J. Tan, V. Kumar, H. Zhao, and S. Levine, "Soft Actor-Critic Algorithms and Applications," *arXiv preprint arXiv:1812.05905*, 2018.

[15] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[16] W. L. Hamilton, R. Ying, and J. Leskovec, "Representation learning on graphs: Methods and applications," *IEEE Data Engineering Bulletin*, vol. 40, no. 3, pp. 52–74, 2017.

[17] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Y. Bharath, and P. W. Battaglia, "Graph networks as learnable physics engines for inference and control," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[18] R. Li, K. Yu, C. Shen, and H. Li, "Modeling physical systems with graph neural networks: A case for robotic manipulation," *arXiv preprint arXiv:2006.04329*, 2020.

[19] L. Kovács and A. Jlidi, "Graphs Unveiled: Graph Neural Networks and Graph Generation," *arXiv preprint arXiv:2403.13849*, Mar. 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2403.13849

[20] M. Bucki, C. Lobos, Y. Payan, et al., "Jacobian-based repair method for finite element meshes after registration," *Engineering with Computers*, vol. 27, pp. 285–297, 2011, doi: 10.1007/s00366-010-0198-2.

[21] A. G. Pandala, Y. Ding and H. -W. Park, "qpSWIFT: A Real-Time Sparse Quadratic Program Solver for Robotic Applications," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3355-3362, Oct. 2019, doi: 10.1109/LRA.2019.2926664.

Authorized licensed use limited to: Princeton University. Downloaded on March 24,2026 at 17:40:15 UTC from IEEE Xplore. Restrictions apply.